

Lofi Generator

A procedural lofi beat engine with no pre-recorded track underneath it, built to prove the harmony myself instead of asking a model to write it

Role	Sole designer and developer
Timeline	1 to 2 weeks, built in 2026
Stack	JavaScript, Web Audio API, Canvas API
Samples	Piano and drum samples from meel-hd/lofi-engine (MIT licensed); everything else, harmony, melody, every ambience bed, synthesized from scratch

Why Generate It Myself

I had seen lots of lofi generators that focused on assembling or generating finished tracks. I wanted to explore the opposite approach: encoding the musical decisions myself, chord theory, melodic phrasing, the mechanics behind why a progression resolves or resists resolution, rather than outsourcing that thinking to a language model and wiring up a player around its output.

This project sits in the same place my art portfolio and my main site architecture already sit: proof that two sides of what I do, music and code in this case, aren't separate hobbies I happen to have, they're the same instinct applied to different material. I'd already combined art and code once. This was the next obvious combination.

What's Actually Generated, Not Sampled

The only pre-recorded audio in the whole system is the piano and drum samples themselves, ported from meel-hd/lofi-engine under its MIT license.¹ Everything that decides what to play, the chord progression, the melody line, every ambience layer (rain, fire, traffic, wind), is generated live, synthesized or algorithmically triggered from scratch. The track is not built around a stored phrase loop; each bar is generated from the current system state against the same rule set, so repeated sections emerge from the system rather than being replayed.

The engine is split into independent generators: harmony produces the current chord progression, melody consumes that harmonic state to choose its next note, ambience runs on its own independent stochastic clocks unrelated to the musical grid, and a lookahead scheduler translates all of it into Web Audio timing.

The Chord Engine: Real Harmony, Weighted Against Repetition

Chord progression isn't a fixed loop cycling through a pre-written sequence. Each chord change is a live decision, constrained by an actual functional-harmony table, the same kind of rule a music theory class teaches for which chords can reasonably follow which, so a I chord can move to a ii, iii, IV, V, or vi, but not to anything that would sound like a harmonic non sequitur.

Inside that constraint, the choice isn't uniform random. Each allowed next chord is weighted by how long it's been since it was last used, a chord that hasn't played yet gets a strong pull toward being chosen, one played two bars ago is deliberately less likely. That does two things at once: it keeps the progression from feeling like it's cycling through an obvious pattern, and it guarantees a chord never repeats twice in a row without needing a separate rule to check for that, since every option already excludes its own index by construction.

A chord never used yet gets a strong boost over one used recently. The rule against repeating a chord twice in a row falls out of the table for free.

The goal wasn't maximum unpredictability. It was avoiding obvious loops while keeping enough harmonic memory that the result still feels intentional.

The Melody: A Random Walk That Still Agrees With the Harmony

Melody notes are chosen relative to the previous note, a small step up or down the scale, rather than each note being picked independently. That's what keeps a random process sounding like a phrase instead of a string of unrelated pitches. But a plain stepwise walk has no awareness of what chord is playing underneath it, and left alone it can land anywhere in the scale regardless of the harmony, which is a specific failure mode worth naming honestly: it sounds bad. Not wrong by any rule, just wrong to the ear.

The fix was giving the first note of each phrase a different rule than the notes that follow it. Instead of a random step, it searches outward from the current position for the nearest note that's actually a tone of the current chord, capped at a four-step search so a distant match doesn't force a melody leap across most of the scale's range. Later notes in the same phrase still use the ordinary small-step walk as passing tones connecting one chord-anchored landing to the next, closer to how an actual improvised phrase behaves than either a fully locked-in melody or a fully free one would be. The melody agrees with the harmony at the moments that matter most and wanders freely in between.

Scheduling: Real Lookahead, and Ambience That Refuses the Grid

Playback runs on a proper lookahead scheduler, the established pattern for timing-accurate Web Audio playback, rather than relying on JavaScript's own timers, which drift under load in a way that would eventually be audible as timing slop.

The ambience layers, rain, thunder, traffic, fire, wind, deliberately don't run on that same musical grid. Each one gets its own independent, irregular clock instead of a per-beat chance to fire. That wasn't the first version. An earlier pass had rain quantized to the same 16th-note grid as everything else, and it read as a mechanical hiss instead of actual drops, because real rain isn't quantized to a tempo and forcing it onto one made it sound like a machine instead of weather. The same tuning happened layer by layer: traffic passes were originally spaced too far apart to read as an ongoing presence rather than a rare event, twig snaps in the forest bed needed to show up more often to be noticeable against the wind bed sharing its frequency range, fireplace crackle needed more density to feel like a fire instead of an occasional pop. None of these were solved by changing the algorithm's logic, they were solved by changing numbers and listening again.

Small Technical Details

- Chord and arpeggio onsets are given a few milliseconds of intentional timing jitter rather than landing exactly on the grid instant. Without it, the chord and the kick drum coincided down to the sample every time, which read as one triggered event instead of two separate instruments landing together on the beat.
- Reverb, saturation, and the pink-noise bed underneath everything are all generated programmatically, an impulse response built at runtime, a saturation curve computed rather than sourced, not loaded from any external effect or plugin.
- A regenerate control rerolls the underlying parameters, tempo, key, harmony density, entirely, producing a different piece rather than restarting the same one.

The Real Difficulty: Tuning by Ear

None of the hardest part of this project was music theory or algorithm design. Music theory gives real rules and real boundaries, but a progression or a melodic choice can be entirely correct by those rules and still land wrong to the ear, jarring, clashing, or just off in a way theory alone doesn't predict. That mismatch showed up constantly through the build.

Fixing it meant listening for hours at a time, noting every clash or overwhelming chord as it happened, and often tracking the same problem down more than once before actually understanding what was causing it rather than just patching the symptom. By the end of a long tuning session, everything started sounding the same, and I was hearing lofi in my dreams. That's not a complaint, it's genuinely what the work was: theory tells you what's allowed, only your own ear tells you what actually sounds right, and there's no shortcut between those two things except spending the time.

Outcome

The generator is live on the site with a full mixer exposed to the listener, independent volume control over chords, melody, and drums, a rain and thunderstorm layer, four selectable atmosphere beds (cityscape, forest, ocean, fireplace), a vinyl crackle toggle for room tone, and a regenerate control. A canvas waveform reacts live while it plays, and the interface surfaces the actual music theory underneath in real time, the current roman-numeral degree, the chord name, and the tempo, updating as the engine makes each decision.

Reflection

What this project actually tested wasn't whether I could write a chord table or a scheduling loop, both of those are learnable in a way that has a clear right answer once you know the rules. What it tested was whether I could trust my own ear over what the rules said should already be correct, and keep listening past the point where everything blurs together, because that's the only way the gap between technically right and actually good ever closes. I went into this specifically to prove that music knowledge could replace a model doing the creative work. What I came out with is a better sense of how much of that knowledge only becomes real once you've spent hours confirming it against something you can actually hear. That ended up being the same lesson as the rest of these projects: constraints can force understanding, but they can't replace judgment.

References

- 1 *meel-hd/lofi-engine*. Piano and drum sample source, MIT License. Used for sampled instrument audio only; harmony, melody, and ambience generation are original to this project.